Lecture 26.09: Logistic Regression from Scratch — Mathematical Formulation, Sigmoid & Log-Loss

📌 Executive Summary

First-principles derivation of Logistic Regression for binary classification, Sigmoid squashing function, Bernoulli likelihood, Log-Loss derivation, and gradient mechanics.

🔬 Mathematical Formulation & Theory

1. The Classification Paradigm Shift

Linear regression predicts continuous values $y \in (-\infty, +\infty)$. Attempting to use linear regression for binary classification ($y \in \{0, 1\}$) introduces fatal flaws:

1. Predictions $\hat{y}$ can be negative or vastly exceed 1, violating probability axioms.
2. An extreme positive outlier shifts the decision line drastically, corrupting classifications for well-separated points.

2. The Sigmoid Activation Function

To bound predictions strictly within probability space $[0, 1]$, Logistic Regression routes the linear output $z = \mathbf{w}^T\mathbf{x} + b$ through the **Sigmoid (logistic) function**:

$$g(z) = \sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

Key Mathematical Properties:

- Symmetry: $\sigma(-z) = 1 - \sigma(z)$
- Midpoint: $\sigma(0) = 0.5$
- First Derivative Identity:

$$\frac{d}{dz}\sigma(z) = \sigma(z)(1 - \sigma(z))$$

3. Bernoulli Likelihood & Binary Cross-Entropy Loss (Log-Loss)

Under the Bernoulli assumption:

$$P(y=1|\mathbf{x}) = \hat{y} = \sigma(z), \quad P(y=0|\mathbf{x}) = 1 - \hat{y}$$

Combined into a single compact probability distribution:

$$P(y|\mathbf{x}) = \hat{y}^{y}(1-\hat{y})^{1-y}$$

Taking the negative log-likelihood across m observations yields the **Binary Cross-Entropy (Log-Loss)** objective:

$$J(\mathbf{w}, b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(\hat{y}^{(i)}) + (1-y^{(i)})\log(1-\hat{y}^{(i)})\right]$$

4. Gradient Derivation — The Mathematical Miracle

Differentiating J with respect to weight w_j via the chain rule:

$$\frac{\partial J}{\partial w_j} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_j}$$

1. $\frac{\partial J}{\partial \hat{y}} = -\left(\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}}\right) = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})}$
2. $\frac{\partial \hat{y}}{\partial z} = \sigma'(z) = \hat{y}(1-\hat{y})$
3. $\frac{\partial z}{\partial w_j} = x_j$

Notice how the term $\hat{y}(1-\hat{y})$ cancels out completely:

$$\frac{\partial J}{\partial w_j} = \left(\frac{\hat{y}-y}{\hat{y}(1-\hat{y})}\right) \cdot [\hat{y}(1-\hat{y})] \cdot x_j = (\hat{y}-y)x_j$$

Summing across all m instances:

$$\nabla_{\mathbf{w}} J = \frac{1}{m} X^T (\hat{\mathbf{y}} - \mathbf{y})$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)})$$

The gradient equations for Logistic Regression are **algebraically identical** to those of Linear Regression, differing only in the definition of $\hat{\mathbf{y}} = \sigma(X\mathbf{w} + b)$!

💻 Python & NumPy Implementation

```
import numpy as np

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def compute_log_loss(y_true, y_pred, eps=1e-15):
    # Clip to prevent log(0) undefined numerical errors
    y_pred = np.clip(y_pred, eps, 1 - eps)
    m = len(y_true)
    loss = - (1.0 / m) * np.sum(y_true * np.log(y_pred) + (1 - y_true) * np.log(1
    return loss

# Demo
z_test = np.array([-5.0, 0.0, 5.0])
probs = sigmoid(z_test)
print("Sigmoid values for [-5, 0, 5]:", probs)
```

🎯 Key Takeaways & Best Practices

1. **First-Principles Transparency**: Building from scratch demystifies optimizer mechanics, memory layouts, and algorithmic failure modes.
2. **Vectorization vs Loops**: Always eliminate Python `for` loops across sample dimensions by relying on BLAS-accelerated NumPy array operations.
3. **Shape Integrity**: Maintain explicit assertions on input array ranks and shapes (`X.shape == (m, d)` , `y.shape == (m,)`) to prevent silent broadcasting bugs.